TikTon — Complete Product Specification

> Feed this document to Claude in plan mode. It contains every detail about what TikTon is, what it does, and how every feature works. Claude should use this to generate the full step-by-step technical implementation plan for the entire project. This is a **hackathon project** — it must be functional and impressive, but not production-grade.

Context & Background

What We're Building

TikTon is a **Telegram Mini App** — a web application that runs inside Telegram's built-in browser. It is a TikTok-style short-form video platform powered by the TON blockchain. Users open it inside Telegram, scroll a full-screen vertical video feed, record or upload short clips, follow creators, and interact through likes, comments, shares, and tips. Creators monetize through Telegram Stars, Toncoin payments, subscriptions, their own fungible tokens, NFT collectibles, and more.

This is a **hackathon project** for the BSA "Stablecoins & Payments" Hackathon, AlphaTON Capital track. It needs to be functional enough for a live demo, impressive enough to win, but does not need to be production-grade or scalable. Simplicity in infrastructure is preferred over enterprise patterns.

Why We're Building It

The AlphaTON Capital track is the generalist TON track — we can build anything as long as it makes sense on TON and can realistically win users. Entries are judged on three things: idea, execution, and product-market fit. The jury is looking for a clear problem and solution, a credible path to adoption through Telegram distribution, and bonus points for Telegram-native feel, clean onboarding, thoughtful use of TON primitives, and a practical monetization or growth angle that could scale.

We spoke to a jury member who told us they're specifically looking for a Telegram app that helps people transition from TikTok to Telegram, offering content creators monetization using Telegram-native features and the TON blockchain.

Prize Pool

- 1st Prize: $3,000 USD + 2,000 TON + Claude Max subscription
- 2nd Prize: $2,000 USD + 1,000 TON + Claude Max subscription
- 3rd Prize: $1,000 USD + 999 TON + Claude Max subscription
- WonTon Soup Prize: 100 TON + special recognition

Market Context

Telegram has 1 billion monthly active users with native wallet support. The TON Foundation's own presentation from Miami outlines a full creator economy toolkit already built into Telegram: ad revenue at 1,000 subscribers, paid private messages to channels, private chats powered on-chain, community collectibles, Stars reactions that convert to real value, Telegram Gifts in TON, paid direct messages, post suggestions via Stars, Mini Apps for communities, and a native affiliate program. The community growth tools include channel giveaways, sellable sticker packs, and gated holders chats (token-gated communities).

Per the Animoca Brands Research report (March 2026), Telegram achieved $1B in revenue in 2024 and is on track for $2B in 2025. Revenue streams include premium subscriptions ($223M in H1 2025), digital goods and Stars ($222M in H1 2025), advertising ($125M in H1 2025), and TON exclusivity deals ($300M in 2025). Stars total purchases reached $160M by 2025. At its 2025 peak, Telegram processed 70 million daily Gift transactions and 6 million daily Sticker transactions. TON Space (the native non-custodial wallet) surpassed 100 million registered users by November 2024.

The TON blockchain handles 100,000+ TPS via dynamic sharding, supports Jettons (fungible tokens, TEP-74 standard), NFTs (TEP-62 standard), and Toncoin serves as the economic backbone for the entire ecosystem.

TikTok faces regulatory bans and restrictions globally. Creators with large audiences face platform risk. Current alternatives (YouTube Shorts, Instagram Reels) impose 30-55% revenue shares. Telegram takes 0% commission on Stars. This is the gap TikTon fills.

What Makes TikTon TON-Native (Not Just Crypto-Bolted-On)

This is critical for the hackathon. The jury will check whether TON is essential or decorative. These features cannot exist without a blockchain:

Creator Tokens (Jettons): Every creator can launch their own fungible token on TON. Fans buy creator tokens to access premium content. The token's value reflects the creator's popularity. Early supporters benefit as the creator grows. This is an on-chain creator economy that fundamentally cannot work without blockchain infrastructure.

On-Chain Content Registration: Every video uploaded to TikTon gets SHA-256 hashed and the hash is stored on a TON smart contract with the creator's address and timestamp. This creates an immutable, verifiable proof of who published content first. It solves TikTok's biggest creator pain point — content theft — in a way that only a blockchain can provide.

NFT Collectibles: Creators mint limited-edition digital goods as TON NFTs following the TEP-62 standard. These are tradeable on secondary markets like Getgems. Ownership is verifiable on-chain.

Token-Gated Access: A smart contract checks whether a user's wallet holds a specific Jetton or NFT before granting access to premium content. This is trustless access control — no platform can override it.

Trustless Subscriptions: Subscription payments flow through a TON smart contract. The creator receives funds directly — no platform can withhold or delay payment.

Toncoin Tipping via Smart Contract: TON tips route through a TipJar contract that enforces the fee split transparently on-chain.

Deployment Target

All TON smart contracts will be deployed to TON testnet. Testnet Toncoin is free (available from faucets). This means contract deployments, content registration, token launches, NFT minting, tipping, and subscriptions cost nothing. TonConnect wallets (Tonkeeper, TON Space) need to be switched to testnet mode for testing.

Telegram Stars are the only thing that costs real money — there is no testnet for Stars. But Stars are cheap (a few dollars is more than enough for all demo interactions).

Feature Priority Tiers

> **This section is critical for implementation planning.** Claude must build features in priority order. All Tier 1 features must be complete and working before starting any Tier 2 feature. All Tier 2 features must be complete before starting any Tier 3 feature. If time runs out, the app is still demoable with only Tier 1 complete.

Tier 1 — Must Ship (The Demo-Winning Features)

These are the features the jury will actually evaluate. They must be fully functional.

1. **Telegram Mini App shell** — opens inside Telegram, zero-friction auth via initData, adapts to Telegram's theme
2. **Video feed** — full-screen vertical swipeable feed with autoplay, For You / Following / Trending tabs
3. **Video upload from gallery** — pick a video from device, add caption and hashtags, publish
4. **In-app camera recording** — record video within the app (with iOS fallback to file picker)
5. **Creator profiles** — avatar, bio, stats, video grid, follow button
6. **Social interactions** — like, comment, follow, share to Telegram chat
7. **Stars tipping** — tip any creator with Telegram Stars via native invoice
8. **Paid content unlock** — creators set a Stars price, viewers pay to unlock
9. **TonConnect wallet integration** — connect/disconnect TON wallet
10. **Toncoin tipping** — tip creators with TON via TipJar smart contract
11. **On-chain content registration** — video hash stored on Content Registry smart contract, "Verified on TON" badge
12. **Creator Jetton launch** — creator deploys their own token via TonConnect
13. **Token-gated content** — videos locked behind Jetton/NFT ownership, verified on-chain

14. **Subscriptions** — pay with TON via Subscription Manager smart contract, unlock subscriber-only content

Tier 2 — Should Have (Polish and Extras, if Time Allows)

These features add polish and demonstrate broader capability. Build after all Tier 1 is complete.

15. **NFT collectibles** — creators deploy NFT collections, fans mint
16. **Creator dashboard** — analytics overview, earnings breakdown, content management
17. **Explore / search page** — search for creators and videos, trending hashtags
18. **Star Reactions** — premium animated reactions on videos (🔥 💎 🚀 👑) paid with Stars
19. **Paid direct messages** — fans pay Stars to DM creators (in-app messaging, not Telegram DMs)
20. **Notifications** — in-app notification list for likes, comments, follows, tips, etc.
21. **Sound library** — browse and add royalty-free music tracks to videos
22. **Bookmarks** — save videos to a personal collection

Tier 3 — Optional / Describe in Pitch Only

These are either too complex for the hackathon timeline or not essential for the demo. Mention in the pitch deck as roadmap items. Only build if Tier 1 and Tier 2 are fully done.

23. **Video editing** — trim, filters, speed adjustment, text overlays
24. **Referral/affiliate tracking** — referral links with commission tracking
25. **Real-time features** — live reaction broadcasts, live comment updates via WebSocket
26. **Advanced recommendation algorithm** — ML-based personalization
27. **Inline bot sharing** — share videos via @TikTonBot inline queries
28. **Live streaming** — not building this, pitch deck only

Prerequisites: Things the Developer Must Provide Manually

> **Critical: Claude cannot generate or create the items below.** When generating the step-by-step implementation plan, Claude must include explicit steps telling the developer when to pause and provide each of these, in the correct order. The plan should not proceed past a step that depends on a prerequisite until the developer has confirmed it's in place.

Before Any Coding Begins

1. **Telegram Bot Token:** The developer must open Telegram, message @BotFather, create a new bot (e.g., `@TikTonBot`), and copy the bot token. This is needed before the backend or

bot can be started. The developer should also set the bot's name, description, and profile picture through BotFather.

2. **TON Wallet for the Platform:** The developer must create a TON wallet (using Tonkeeper or TON Space) that will serve as the platform's backend wallet — used by the server to send on-chain transactions like registering content hashes. The developer must export the 24-word mnemonic seed phrase. Switch this wallet to **testnet mode** and get free testnet Toncoin from a faucet (e.g., t.me/testgiver_ton_bot).
3. **TON Center API Key:** The developer must go to toncenter.com, register, and get a free API key for **testnet**. This is needed for the backend to query the TON blockchain.

Before Backend Can Run

4. **Object Storage (S3-Compatible):** The developer must create a storage bucket for video and image files. Recommended: Cloudflare R2 (generous free tier, S3-compatible). The developer needs: endpoint URL, bucket name, access key ID, and secret access key. This is where uploaded videos and thumbnails are stored.

Before Frontend Deployment

5. **App Icon / Logo:** The developer must provide a logo image for TikTon (at least 256x256 PNG). This is used in the TonConnect manifest file and as the bot's profile picture. A simple logo from Canva is fine.

Before Demo

6. **Sound Files (only needed for Tier 2, feature #21):** The developer must source 10-15 royalty-free MP3 audio files (from Pixabay Music, Free Music Archive, Uppbeat, or similar) and place them in `assets/sounds/` along with a `sounds.json` manifest file. See the Sound Library section below for the exact format.
7. **Mini App Registration:** After the frontend is deployed to a public URL, the developer must go back to @BotFather in Telegram and configure the Mini App: send `/newapp`, select the bot, provide the app title, and set the web app URL to the deployed frontend URL. This makes the Mini App accessible via `https://t.me/TikTonBot/app`.
8. **Demo Content:** After the app is fully deployed and functional, the developer should record and upload 15-20 videos through the app itself to populate the feed before the demo. Include a mix: some public videos, a few paid/locked ones, and at least one creator account with a launched token. Use a second Telegram account to test the viewer experience.

Environment Variables

All of the above items translate into environment variables. The plan should create a `.env.example` early on:

- `BOT_TOKEN` — from BotFather (step 1)
- `BOT_USERNAME` — the bot's username without @ (step 1)
- `MINI_APP_URL` — deployed frontend URL (step 7, leave blank until deployment)
- `S3_ENDPOINT` — storage endpoint (step 4)
- `S3_BUCKET` — bucket name (step 4)
- `S3_ACCESS_KEY` — storage access key (step 4)
- `S3_SECRET_KEY` — storage secret key (step 4)
- `S3_REGION` — storage region, use "auto" for R2 (step 4)
- `TON_NETWORK` — always "testnet" for this project
- `TON_API_KEY` — from toncenter.com testnet (step 3)
- `PLATFORM_WALLET_MNEMONIC` — 24-word seed phrase (step 2)
- `JWT_SECRET` — any random string, Claude can generate this
- Contract addresses — filled in after smart contract deployment, Claude handles this

---

The App: Complete Feature Description

1. Onboarding

1.1 Viewer Onboarding (Target: Under 5 Seconds)

When a user taps a TikTon link or opens the bot in Telegram, the **Telegram Mini App** launches inside Telegram's built-in browser. There is no sign-up form, no email field, no password creation, no verification step. Telegram's WebApp SDK provides `initData` which contains the user's Telegram ID, first name, last name, username, and profile photo URL. The backend validates this data cryptographically (HMAC verification using the bot token) and either finds the existing user or creates a new one. A session token is returned and stored in memory for the session.

The user lands directly on the For You video feed. They can immediately start watching, liking, commenting, and following creators without any wallet or payment setup.

After watching their third video, a small, dismissible banner appears at the bottom: "Connect your TON wallet to send tips and unlock rewards." This is purely optional. Viewers can use the entire app (browse, like, comment, follow, share) without ever connecting a wallet. The wallet is only needed for TON-based transactions (Toncoin tips, subscriptions, buying creator tokens, minting NFTs, accessing token-gated content). Stars-based transactions (Stars tips, unlocking paid content, Star Reactions, paid DMs) work through Telegram's native payment system and don't require a wallet at all.

1.2 Creator Onboarding (Target: Under 2 Minutes)

Any user can become a creator. They tap the "+" button in the bottom navigation bar or tap "Become a Creator" from their profile settings. If they haven't connected a TON wallet yet, they're prompted to do so via TonConnect 2.0 — this is a single-tap confirmation in their wallet app (TON Space, Tonkeeper, etc.). The wallet connection is required for creators because they need it to receive Toncoin payments, deploy tokens, and deploy NFT contracts.

After wallet connection, a profile setup screen appears. Display name, bio, and profile picture are pre-filled from their Telegram profile. They can edit anything. They then configure their initial monetization settings: Stars tips are enabled by default, and they can optionally set a price for paid DMs and configure subscription tiers. All of this can be changed later.

Once setup is complete, the user is marked as a creator and can upload videos.

---

2. Video Feed & Discovery

2.1 For You Page (FYP)

This is the main screen of the app. It occupies the full screen within the Telegram Mini App — no visible browser chrome, no header bar reducing viewport. Each video fills 100% of the viewport height and width. Videos play in a vertical, full-screen format (9:16 aspect ratio), identical to TikTok.

The user swipes up to advance to the next video and swipes down to go back to the previous one. Videos autoplay immediately when they come into view. On the very first load (before any user interaction), videos start muted due to browser autoplay policies. After the user taps once anywhere on the screen, sound activates and all subsequent videos play with sound on.

Single-tapping the video toggles play/pause. A semi-transparent play icon appears briefly in the center when paused. Double-tapping anywhere on the video triggers a like, with an animated heart burst effect at the tap location.

The right side of the screen has a vertical action bar with the following buttons from top to bottom:

- Creator's avatar (circular, tappable — navigates to their profile)
- Heart icon + like count (toggles like on tap)
- Comment bubble icon + comment count (opens comment sheet)
- Share/arrow icon (opens Telegram's share dialog)
- Star icon (opens the tip modal)
- Bookmark icon (saves video to bookmarks) — Tier 2

The bottom of the screen overlays the video with:

- Creator's display name (bold, tappable — navigates to profile)
- Caption text (first 2 lines visible, "more" to expand, hashtags are colored and tappable)

- Sound/music name with a small music note icon (if a sound was added) — Tier 2

At the very top of the screen are three tabs: "Following", "For You", and "Trending". The user can tap to switch between them.

Feed Algorithm (hackathon-grade, kept simple): The For You feed uses a simple scoring algorithm. Each video's score is calculated based on: likes (weight 3), comments (weight 5), shares (weight 8), minus a decay factor based on age in hours (subtract 0.5 per hour). The feed returns videos sorted by this score. No ML, no per-user personalization beyond following — just a simple global ranking that produces a reasonable feed.

Following Feed: Chronologically ordered videos from creators the user follows. Newest first.

Trending Feed: Videos with the highest engagement scores from the last 24 hours.

2.2 Explore / Search Page (Tier 2)

Accessible from the bottom navigation bar. A search bar at top. Below it, trending hashtags as tappable pills. Search results appear as: creators (list with avatar, name, follower count) and videos (thumbnail grid). Simple text search on captions and usernames.

---

3. Content Creation

3.1 In-App Camera Recording

When the user taps the "+" (create) button in the bottom navigation, they enter the creation flow. The first screen is the camera view.

The app requests camera and microphone permissions from the device. If granted (which works on most Android devices and some iOS versions within Telegram's WebView), the user sees a live full-screen camera preview. The preview is mirrored when using the front camera.

Controls on the camera screen:

- **Record button** (large, centered at bottom): Tap once to start recording, tap again to stop. The button pulses/animates while recording.
- **Flip camera button**: Switches between front and rear camera.
- **Duration indicator**: Shows elapsed recording time. Maximum recording length is 3 minutes (180 seconds).

When the user finishes recording and taps stop, they see a preview of their recorded video with "Use this clip" and "Retake" options.

iOS WebView Fallback: If the camera API is not available (which happens on some iOS versions inside Telegram's WebView), the app gracefully falls back to a file picker that opens the device's native camera app. The user records there, and the video is imported back into TikTon. This fallback must be seamless — the user sees a "Record Video" button that opens the native camera instead of an error message.

3.2 Video Upload from Gallery

Alongside the camera option, there's an "Upload" button that opens the device's file picker filtered to video files. The user selects a video from their gallery. Client-side validation checks: maximum file size is 100MB, maximum duration is 180 seconds, and the file must be a valid video format (MP4, WebM, QuickTime). After selection, a preview plays so the user can confirm.

3.3 Sound Library (Tier 2)

A "Sounds" button on the creation screen opens a sheet where the user can browse music tracks. They can preview tracks by tapping them and select one to add as background audio to their video.

Important note on the sound library: The audio files cannot be generated by AI — they must be manually sourced. The developer will place 10-15 royalty-free MP3 files (sourced from Pixabay Music, Free Music Archive, or similar) into an `assets/sounds/` folder in the repository, along with a `sounds.json` manifest file listing each track's metadata. The format for each entry: `{ "id": "s1", "title": "Viral Beat #1", "artist": "Unknown", "genre": "pop", "duration": 32, "file": "viral-beat-1.mp3" }`. Pick a mix of genres: upbeat pop, lo-fi chill, EDM, phonk, comedic, cinematic. Keep tracks short (30-60 second loops). Give the tracks catchy, TikTok-style display names (e.g., "Main Character Energy", "Plot Twist", "CEO Walk") rather than generic filenames.

3.4 Video Editing (Tier 3 — Optional)

If time allows after all Tier 1 and Tier 2 features are complete: trim tool with start/end handles, CSS filter presets (Warm, Cool, Vintage, B&W, Vivid), speed adjustment (0.5x, 1x, 2x). These are nice-to-have polish features. Do not build these at the expense of core features.

3.5 Publishing Flow

After recording or selecting a video, the user goes to the publish screen:

Caption: Text input, max 500 characters. Hashtags typed with "#" are automatically extracted and stored for search and feed purposes.

Visibility setting (this is crucial for monetization):

- **Public** — Anyone can see this video. Default option.
- **Subscribers Only** — Only users with an active paid subscription to this creator can see it. Non-subscribers see a locked thumbnail.
- **Token-Gated** — Only users who hold a minimum amount of the creator's Jetton token or own an NFT from a specified collection can view it. When selected, the creator chooses which token/NFT and the minimum balance.
- **Paid (Stars)** — Anyone can see a 3-second blurred preview, but must pay a specified Stars amount to unlock. Creator sets the price.

Register on TON blockchain: Toggle, enabled by default. The video's file hash will be stored on the TON Content Registry smart contract after upload, creating an immutable proof of authorship.

Allow comments: Toggle, on by default.

When the user taps "Post", the video uploads to storage, a thumbnail is generated, the hash is computed, and the video appears in the feed. If on-chain registration is enabled, the hash is stored on TON after upload completes. The user sees an upload progress indicator and a "Posted!" confirmation.

---

4. Monetization Features

This is TikTon's core differentiator. Every monetization path uses either Telegram Stars (0% Telegram commission) or Toncoin. Creators keep the vast majority of their earnings.

4.1 Stars Tipping (Tier 1)

Every video has a star/tip button in the action bar. Tapping it opens a tip modal as a bottom sheet with two tabs: "Stars" and "TON".

On the Stars tab, preset tip amounts are displayed as tappable buttons: 1 ⭐, 5 ⭐, 10 ⭐, 50 ⭐, 100 ⭐, and a "Custom" button. When the user selects an amount, Telegram's native payment interface opens (via the Mini App's `openInvoice` method). The user confirms the Stars payment within Telegram's own UI. After payment succeeds, the creator is credited immediately.

A floating animation appears over the video: "⭐ +50" with the tipper's name. The creator's earnings update in their profile/dashboard.

Revenue split: The platform takes 5%, the creator keeps 95%. Telegram takes 0%.

4.2 Toncoin Tipping (Tier 1)

On the TON tab of the tip modal, preset amounts are: 0.1, 0.5, 1, 5, 10 TON, and Custom. This requires the user to have a connected TON wallet. When they select an amount, a TonConnect transaction request appears in their wallet app. They confirm. The tip is routed through the TipJar smart contract on TON testnet, which enforces the 95/5 split on-chain: 95% to the creator's wallet, 5% to the platform wallet. The transaction is verifiable on the TON testnet explorer.

If the user hasn't connected a wallet, tapping the TON tab shows a "Connect Wallet" button.

4.3 Paid Content / Stars Paywall (Tier 1)

When a creator publishes a video with "Paid" visibility and sets a Stars price, non-paying viewers see:

- A 3-second blurred/frosted preview that loops
- A centered lock icon

- "Unlock for X ⭐ " text and button

Tapping unlock triggers the same Stars payment flow as tipping. After payment, the video unlocks permanently for that user. The lock overlay dissolves and the full video plays.

On the creator's profile grid, paid videos show a small star badge on their thumbnail.

4.4 Subscriptions via TON Smart Contract (Tier 1)

Creators configure subscription tiers on their profile. Up to three tiers with customizable names and prices in TON (e.g., Bronze: 2 TON/month, Silver: 5 TON/month, Gold: 10 TON/month).

When a fan taps "Subscribe", a TonConnect transaction for the corresponding TON amount goes to the Subscription Manager smart contract on testnet. The contract records: subscriber address, creator address, tier, and expiry timestamp (current time + 30 days).

After subscribing:

- The fan sees a "Subscribed" badge on the creator's profile
- All "Subscribers Only" videos are unlocked
- The fan gets a subscriber badge next to their name in comments on that creator's videos

When the subscription expires (30 days after payment), access is revoked. No auto-renewal — the fan renews manually.

The creator can withdraw accumulated subscription revenue at any time via TonConnect.

4.5 Creator Tokens / Jetton Launch (Tier 1)

From their profile, the creator taps "Launch Your Token" and fills out:

- Token name, symbol, total supply
- Price per token in TON
- Creator allocation percentage (minted to creator wallet on deploy)
- Token icon (upload image)

On submit, a TonConnect transaction deploys a Jetton Minter contract on TON testnet. The creator pays testnet gas (free). Once deployed, the token is live.

The creator's profile now shows a "Token" section with: token name, symbol, price, and a "Buy" button. Fans buy tokens via TonConnect. Tokens are standard TEP-74 Jettons — transferable and tradeable.

4.6 On-Chain Content Registration (Tier 1)

When a creator publishes a video with "Register on TON" enabled:

1. The video file is hashed using SHA-256

2. The hash + creator address + video ID are sent to the Content Registry smart contract on TON testnet
3. The contract stores this record permanently. It enforces uniqueness — the same hash cannot be registered twice.
4. The video displays a "Verified on TON ✓" badge with a link to the transaction on a TON testnet explorer

This proves who published content first, solving content theft in a blockchain-native way.

4.7 Token-Gated Content (Tier 1)

When a creator publishes a video with "Token-Gated" visibility, they select which asset is required (their Jetton token and minimum balance, or an NFT collection).

When a viewer encounters a token-gated video:

- If no wallet connected: blurred preview + "Connect your wallet to view"
- If wallet connected but insufficient balance: blurred preview + "You need X $TOKEN to view" + "Buy Tokens" button linking to creator profile
- If wallet connected and holds enough: video plays normally

The check is done by the backend querying the TON testnet for the viewer's Jetton wallet balance or NFT ownership.

4.8 NFT Collectibles (Tier 2)

Creators deploy NFT collections (TEP-62) from their profile. They set: name, description, image, max supply, mint price in TON, royalty percentage.

On submit, TonConnect deploys the NFT Collection contract on testnet. Fans mint from the creator's profile → TonConnect transaction → NFT Item deployed, owned by fan. NFTs can be viewed on any TON testnet explorer.

4.9 Star Reactions (Tier 2)

While watching a video, the user can send a premium animated reaction costing Stars. Access by long-pressing the like button. Options: 🔥 (1⭐), 💎 (5⭐), 🚀 (10⭐), 👑 (50⭐). Stars payment is processed, the reaction emoji animates across the screen. Revenue goes to creator (95/5 split).

4.10 Paid Direct Messages (Tier 2)

Creators set a DM price (in Stars) on their profile. When a fan taps "Message", they see a notice about the cost. Sending triggers a Stars payment. Messages are stored within TikTon's system (in-app messaging, not actual Telegram DMs). The creator sees messages in an inbox section. Creators reply for free.

4.11 Affiliate / Referral (Tier 3 — Optional)

Referral links via Telegram deep link parameters. When a referred user signs up and transacts, the referrer earns 10% of platform fees for 90 days. Only build if time allows.

---

5. Social & Community Features

5.1 Likes (Tier 1)

Tapping the heart icon or double-tapping the video toggles a like. Double-tap produces an animated heart burst effect at the tap location. Like count updates immediately in the UI. A notification is created for the video creator.

5.2 Comments (Tier 1)

Tapping the comment icon opens a bottom sheet covering about 60% of the screen. The video continues playing behind it. The sheet contains:

- A scrollable list of comments: avatar, username, text, timestamp (e.g., "2h ago"), small like button per comment
- Nested replies (1 level): tap "Reply" → input prefills with @username, reply appears indented under parent
- An input bar at the bottom with text field and send button

Posting a comment creates a notification for the video creator.

5.3 Sharing (Tier 1)

Tapping the share icon triggers Telegram's native share dialog via the Mini App SDK. The user can share the video link to any Telegram chat. The link is a deep link: `https://t.me/TikTonBot/app?startapp=v_VIDEOID`. When the recipient opens it, the Mini App navigates directly to that video. There's also a "Copy Link" option.

5.4 Following (Tier 1)

On any creator's profile, a "Follow" button. Tapping follows the creator, button changes to "Following". A notification is created for the creator. The "Following" tab in the feed shows content from followed creators.

5.5 Bookmarks (Tier 2)

Bookmark icon on videos saves to a personal collection. Accessible from the user's profile under a "Saved" tab. Bookmarks are private.

5.6 Notifications (Tier 2)

A notification bell with unread count badge. Notification list shows: "X liked your video", "X commented...", "X started following you", "X tipped you 50 ⭐", "X subscribed...", "X bought 100 $YOURTOKEN", etc. Each with actor avatar, text, timestamp, and relevant thumbnail.

Notifications are fetched from the database when the user opens the notifications page (simple polling, not real-time push). No WebSocket needed — the notifications page fetches fresh data each time it's opened or can poll every 30 seconds.

---

6. Creator Dashboard (Tier 2)

Accessible from the creator's profile via a "Dashboard" button.

6.1 Analytics Overview

- Total video views
- Total earnings (Stars + TON combined, with per-source breakdown)
- Follower count and subscriber count
- Top 5 videos by views
- Top 5 videos by earnings

6.2 Content Management

- Grid of all uploaded videos with thumbnail, views, likes, earnings, visibility status
- Tap to edit caption, change visibility, or delete

6.3 Earnings & Withdrawal

- Current Stars balance and current TON balance
- Transaction history
- "Withdraw TON" button → triggers TonConnect transaction to call contract's withdraw function

6.4 Token & NFT Management

- If token launched: name, symbol, contract address, holders count, total earned
- If not launched: "Launch Your Token" button
- List of NFT collections deployed, items minted, total earned (Tier 2)
- "Create New Collection" button (Tier 2)

---

7. User Profile

7.1 Own Profile

- Avatar (from Telegram), display name, bio (editable)
- Stats: Following | Followers | Total likes received

- Wallet connection status (green dot if connected, truncated address)
- Tabs: Videos (grid) | Saved (Tier 2) | Liked
- Token holdings (if wallet connected): list of creator Jettons held with balances
- "Edit Profile" button
- "Dashboard" button (if creator, Tier 2)

7.2 Other User's Profile

- Same layout for public info
- "Follow" / "Following" button
- If creator:
  - Subscription tiers with "Subscribe" button
  - Token section with "Buy" button
  - Collectibles tab with "Mint" buttons (Tier 2)
  - "Message" button with cost (Tier 2)
- Videos tab shows public videos + locked thumbnails for paid/gated content the viewer hasn't unlocked

---

8. Navigation Structure

Bottom navigation bar with 5 items:

1. **Home** (house icon): Main video feed with For You / Following / Trending tabs
2. **Explore** (compass icon): Search and discover (Tier 2)
3. **Create** (plus icon, larger/centered): Recording + upload flow
4. **Inbox** (bell icon): Notifications (Tier 2) and messages (Tier 2)
5. **Profile** (person icon): User's own profile

The bottom nav hides during full-screen video feed viewing and reappears on other pages.

For Tier 1, the Explore tab can show a simple placeholder or redirect to feed, and the Inbox tab can show a "Coming soon" state or a basic notification list. They should be wired up as navigation targets from the start so the structure is in place.

---

9. Deep Linking & Sharing

TikTon supports Telegram deep links:

- **Video link:** `https://t.me/TikTonBot/app?startapp=v_VIDEOID` — opens to a specific video

- **Creator link:** `https://t.me/TikTonBot/app?startapp=u_USERID` — opens to a creator's profile
- **Referral link:** `https://t.me/TikTonBot/app?startapp=ref_USERID` — records referral (Tier 3)

The Mini App parses the `startapp` parameter from `WebApp.initDataUnsafe.start_param` on launch and navigates accordingly.

---

10. Telegram Bot

The TikTon Telegram bot is a simple bot that serves as:

- **Entry point:** The `/start` command sends a message with an "Open TikTon" button that launches the Mini App
- **Payment processing:** Creates Stars invoices and handles successful payment callbacks (pre-checkout queries and successful payment events)
- **Deep link routing:** Parses `startapp` parameters to route users to content

The bot does not need to be complex. It primarily exists for the `/start` command and to process Stars payments. The bot and backend can run in the same process for simplicity.

---

11. Telegram Theme Integration

The Mini App adapts to Telegram's current theme. Telegram provides theme parameters (background color, text color, hint color, link color, button color, secondary background color) via the WebApp SDK. TikTon reads these on load and maps them to CSS variables so the UI matches the user's Telegram appearance — light mode, dark mode, or custom theme. This should be implemented from the very start so the app always looks native.

---

12. Platform Revenue Model

Revenue Stream	Platform Takes	Creator Keeps
Stars Tips	5%	95%
Star Reactions	5%	95%
Paid Content Unlocks (Stars)	8%	92%
Paid DMs (Stars)	5%	95%
Subscriptions (TON)	10%	90%

Revenue Stream	Platform Takes	Creator Keeps
Creator Token Sales (TON)	5% mint fee	95%
NFT Primary Sales (TON)	5% mint fee	95%
NFT Secondary Sales (TON)	2.5% royalty	Creator sets additional royalty

---

13. Technical Approach (For Plan Mode)

> **This section tells Claude what kind of architecture to use. Keep it simple — this is a hackathon.**

This is a Telegram Mini App. The frontend is a web app loaded inside Telegram's WebView. It must use the Telegram Mini App SDK (`@twa-dev/sdk` or the raw `window.Telegram.WebApp` object) for authentication, theme integration, and native features like `openInvoice`.

The project should be structured as a monorepo with the following packages:

1. **Frontend:** React + TypeScript with Vite. Telegram Mini App SDK for auth/theme. TonConnect UI React for wallet integration. Tailwind CSS for styling. Zustand for state. Framer Motion for animations. This is a single-page application.
2. **Backend:** Node.js with Express or Fastify. **SQLite database** (via better-sqlite3 or Prisma with SQLite provider) — no need for PostgreSQL, this is a hackathon. S3-compatible storage (Cloudflare R2) for video/image files. The backend handles API routes, auth validation, database operations, and TON blockchain interactions. The Telegram bot runs in the same process. **No Redis, no job queues, no WebSocket.** Video processing (thumbnail generation) can be done synchronously or with a simple in-process approach. On-chain registration can be triggered directly after upload completes. Keep it simple.
3. **Smart Contracts:** TON smart contracts written in Tact using the Blueprint framework. Contracts: Content Registry, TipJar, Subscription Manager, Jetton Minter + Jetton Wallet (TEP-74), and optionally NFT Collection + NFT Item (TEP-62) for Tier 2. All deployed to **TON testnet**.
4. **Deployment:** Frontend on Vercel or Cloudflare Pages (free). Backend on Railway (free tier with volume mount for SQLite persistence) or any cheap VPS. SQLite database lives on the backend server's filesystem — no external database service needed. The backend must have a public URL so the frontend can call its API and the bot webhook works.

SQLite persistence: Some hosting platforms like Railway use ephemeral filesystems — the SQLite file is lost on server restart. To solve this simply, the backend should back up the SQLite database file to the same S3/R2 storage bucket used for videos. On every database write (or every few minutes), upload the SQLite file to R2 as `backup.sqlite`. On server startup, check if `backup.sqlite` exists on R2 — if it does, download it and use it as the database. This way the

database survives restarts with minimal code. The video files themselves are already safe on R2 regardless.

Key simplifications for hackathon:

- SQLite instead of PostgreSQL — no external database to provision, the database file lives on the server
- No Redis — not needed, cache in memory if absolutely necessary
- No WebSocket — notifications use simple polling (fetch on page load)
- No job queue — process everything synchronously or with simple in-process setTimeout
- Bot and backend run in the same Node.js process — no separate service
- No FFmpeg dependency if possible — generate thumbnails client-side using a canvas screenshot of the video's first frame. Serve the original uploaded video file directly — no server-side transcoding. If the hosting environment supports FFmpeg, use it for thumbnails. If not, skip it.
- Notifications are pull-based (user fetches when they open the page), not push-based
- All TON interactions on **testnet** — zero cost

Communication flow:

- Frontend ↔ Backend: REST API with JWT auth (JWT derived from validated Telegram initData)
- Frontend ↔ TON: TonConnect for user-initiated transactions (tips, subscriptions, deploys)
- Backend → TON: @ton/ton SDK for server-initiated transactions (content registration) and queries (balance checks, subscription checks)
- Bot ↔ Backend: Same process, direct function calls — no HTTP between them

CORS: The frontend (on Vercel/Cloudflare Pages) and backend (on Railway/VPS) will be on different domains. The backend must enable CORS to allow requests from the frontend's domain. Include this in server setup from the start.

Local development: Telegram Mini Apps cannot load from `localhost`. During development, either use ngrok (or a similar tunneling tool) to expose the local dev server via a public HTTPS URL and point BotFather to that, or deploy the frontend early to Vercel and iterate there. The React app can also be developed and tested in a regular browser first (most features work outside Telegram except the WebApp SDK calls), then tested inside Telegram once deployed.

---

14. Technical Details Claude Needs to Know

> **This section contains specific technical facts that Claude may not know or may get wrong. Reference these when implementing the relevant features.**

Telegram Stars currency code is `"XTR"`. When creating a Stars invoice via the Bot API's `createInvoiceLink`, the currency parameter must be `"XTR"` and the provider_token must be an empty string `""`. The prices array uses integer amounts (e.g., 50 means 50 Stars). Example: `{ title: "Tip", description: "Tip for creator", payload: JSON.stringify({type:'tip', videoId, creatorId}), provider_token: "", currency: "XTR", prices: [{ label: "Tip", amount: 50 }] }`.

Checking a Jetton (token) balance is a two-step process on TON:

1. First, query the Jetton Minter contract's `get_wallet_address` getter, passing the user's wallet address. This returns the address of the user's Jetton Wallet contract (each holder has their own wallet contract).
2. Then, query that Jetton Wallet contract's `get_wallet_data` getter to get the balance. If step 1 returns an address that doesn't exist on-chain (contract not deployed), it means the user has zero balance of that token. Handle this case gracefully — it's not an error.

Checking NFT ownership: Query the NFT Collection contract's `get_nft_address_by_index` for each index from 0 to the collection's `next_item_index`, then check each NFT Item's `get_nft_data` to see if the owner matches the user's wallet. For hackathon simplicity, an alternative approach is to store minting events in the database and check ownership there rather than querying the chain for every item.

TonConnect manifest: The TonConnect manifest file (`tonconnect-manifest.json`) must be hosted at a publicly accessible URL. It contains `{ "url": "https://your-app-url.com", "name": "TikTon", "iconUrl": "https://your-app-url.com/icon.png" }`. This is what wallet apps show when asking the user to approve a connection. It must be served from the same domain as the frontend.

Telegram initData validation: The backend must validate `initData` using HMAC-SHA256. The process is: parse the initData query string, remove the `hash` parameter, sort remaining parameters alphabetically, join as `key=value` pairs separated by newlines, compute HMAC-SHA256 using a secret key (which is itself HMAC-SHA256 of the bot token with the key `"WebAppData"`), and compare to the provided hash. This is the only authentication mechanism — there are no passwords or OAuth flows.

Database schema guidance: The database needs at minimum these core tables: `users` (telegram_id, name, username, photo_url, bio, is_creator, wallet_address, stars_balance, dm_price, jetton_address, jetton_name, jetton_symbol), `videos` (creator_id, video_url, thumbnail_url, caption, hashtags, visibility, star_price, required_token, content_hash, registration_tx, view/like/comment/share counts, status), `likes` (user_id, video_id, unique constraint), `comments` (user_id, video_id, parent_id, text), `follows` (follower_id, following_id, unique constraint), `video_unlocks` (user_id, video_id, amount_paid), `tips` (tipper_id, creator_id, video_id, amount, currency, tx_hash), `subscriptions` (subscriber_id, creator_id, tier, ton_amount, tx_hash, expires_at), `notifications` (user_id, type, data_json, is_read), and optionally for Tier 2: `messages` (sender_id, receiver_id, text, stars_paid), `bookmarks` (user_id,

video_id), `nft_collections` (creator_id, name, image_url, contract_address, max_supply, mint_price, minted_count). All tables should have `created_at` timestamps. Use auto-incrementing integer IDs or CUIDs.

---

15. Empty States & First-Run Experience

> **The jury will open the app fresh. These states must look intentional, not broken.**

Empty feed (no videos yet): Show a friendly illustration or icon with text: "No videos yet — be the first to post!" with a prominent "Create Video" button. Do NOT show a loading spinner that never resolves or a blank white screen.

Empty creator profile (no videos): Show the creator's info and stats (all zeros is fine), with an empty grid area that says "No videos yet" and a "Upload your first video" button.

Empty search results: "No results found for [query]. Try a different search."

Empty notifications: "No notifications yet. Start interacting to see activity here!"

Empty messages inbox: "No messages yet."

No wallet connected (on features that need it): Show a clear "Connect Wallet" button with a brief explanation: "Connect your TON wallet to send tips, subscribe, and access exclusive content."

These empty states are important — they prevent the app from looking broken during initial setup or demo preparation.

This document describes every feature, interaction, monetization path, and integration of TikTon. It is designed for a hackathon — functional and impressive, not production-grade. Build Tier 1 features first, then Tier 2, then Tier 3 if time allows. Use this to generate the complete step-by-step technical implementation plan.